COSINE sample code

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def cosine_sim(text1, text2):
    vectorizer = TfidfVectorizer(stop_words="english")
    tfidf = vectorizer.fit_transform([text1, text2])
    return cosine_similarity(tfidf[0:1], tfidf[1:2])[0][0]

# Example
t1 = "I LOVE MACHINE LEARNING"
t2 = "I ENJOY STUDYING MACHINE LEARNING"

print("Cosine Similarity:", cosine_sim(t1, t2))
```

```
Cosine Similarity: 0.4112070550676187
```

JACCARD sample code

```
import re

def tokenize(text):
    return set(re.findall(r"\b\w+\b", text.lower()))

def jaccard_sim(text1, text2):
    set1, set2 = tokenize(text1), tokenize(text2)
    intersection = set1.intersection(set2)
    union = set1.union(set2)
    return len(intersection) / len(union) if union else 0

# Example
print("Jaccard Similarity:", jaccard_sim(t1,t2))
```

```
Jaccard Similarity: 0.5
```

WORDNET sample code

```
import nltk
from nltk.corpus import wordnet as wn
```

```
from nltk.corpus import stopwords

# Run once
nltk.download('wordnet')
nltk.download('stopwords')
nltk.download('omw-1.4')

stop_words = set(stopwords.words("english"))

def get_synsets(text):
    words = [w for w in tokenize(text) if w not in stop_words]
    synsets = []
    for word in words:
        syn = wn.synsets(word)
        if syn:
            synsets.append(syn[0])  # first sense (simple heuristic)
    return synsets

def wordnet_similarity(text1, text2):
    synsets1 = get_synsets(text1)
    synsets2 = get_synsets(text2)

    scores = []
    for s1 in synsets1:
        for s2 in synsets2:
            sim = s1.wup_similarity(s2)
            if sim is not None:
                scores.append(sim)

    return sum(scores) / len(scores) if scores else 0

# Example
print("WordNet Semantic Similarity:", wordnet_similarity(t1, t2))
```

```
WordNet Semantic Similarity: 0.3550149605296664
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
```

STEP 1 - Open a google colab

STEP 2 — Import required libraries

Import libraries for:

• text preprocessing

• similarity calculations

• WordNet semantic similarity

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import nltk
from nltk.corpus import wordnet as wn
from nltk.corpus import stopwords
nltk.download('wordnet')
nltk.download('stopwords')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
True
```

Load or prepare dataset

Students may: • manually create sentence pairs • load from .txt / .csv

They must: • display sample of dataset • explain dataset in 5–6 lines

```
import pandas as pd

dataset = pd.read_excel("sentence_pairs.csv.xlsx")

# show sample
print(dataset.head())
```

```
                        sentence 1                                sentence 2
0          I love machine learning  I enjoy studying artificial intelligence
1    the doctor treated the patient       the physician cured the sick person
2             he bought a new house                   he purchased a new home
3  the teacher explained the lesson           the instructor taught the class
4          the child is eating food                  the kid is having a meal
```

Preprocess text

Write functions to: • lowercase

• remove punctuation & numbers

• remove stopwords

• tokenize

• (optional) lemmatize

```
    def to_lowercase(text):
        """
        Converts all characters in the text to lowercase.
        """
        return text.lower()
    import re

    def remove_punctuation_numbers(text):
        """
        Removes punctuation marks and numeric characters from the text.
        """
        return re.sub(r"[^a-zA-Z\s]", "", text)
    def tokenize(text):
        """
        Splits the text into individual words (tokens).
        """
        return text.split()
    from nltk.corpus import stopwords

    stop_words = set(stopwords.words("english"))

    def remove_stopwords(tokens):
        """
        Removes common stopwords from the list of tokens.
        """
        return [word for word in tokens if word not in stop_words]
```

Represent text numerically

Students must: • choose Bag-of-Words OR TF-IDF

• justify choice in 3–4 lines

Construct matrices suitable for similarity calculations.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Sample sentences extracted from the dataset
sentences = pd.concat(
    [dataset["sentence 1"], dataset["sentence 2"]]
).tolist()

# Initialize TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer(stop_words="english")

# Convert text data into TF-IDF numerical matrix
tfidf_matrix = tfidf_vectorizer.fit_transform(sentences)

# Display matrix shape
print("TF-IDF Matrix Shape:", tfidf_matrix.shape)
```

```
TF-IDF Matrix Shape: (20, 57)
```

Compute Cosine Similarity

Students should: • compute cosine similarity for all text pairs

• print similarity scores

• interpret samples:

Example interpretation: Higher score = more similar meaning Students must explain at least 5 results.

```
from sklearn.metrics.pairwise import cosine_similarity

# Compute cosine similarity matrix
cosine_sim_matrix = cosine_similarity(tfidf_matrix)

# Print similarity scores for all sentence pairs
print("Cosine Similarity Matrix:\n")
for i in range(len(sentences)):
    for j in range(len(sentences)):
        print(f"Similarity between Sentence {i+1} and Sentence {j+1}: "
              f"{cosine_sim_matrix[i][j]:.3f}")
    print()
```

Show hidden output

Double-click (or enter) to edit

```
def jaccard_similarity(tokens1, tokens2):
    """
    Computes Jaccard similarity between two sets of tokens.
    """
    set1, set2 = set(tokens1), set(tokens2)
    intersection = set1.intersection(set2)
    union = set1.union(set2)
    return len(intersection) / len(union) if union else 0
```

Compute Jaccard Similarity

Students should: • compute Jaccard similarity

• compare results with cosine similarity

• interpret at least 5 pairs

Focus on: how overlapping words affect Jaccard score.

```
def preprocess_text(text):
    """
    Applies a series of preprocessing steps to the text.
    """
    text = to_lowercase(text)
    text = remove_punctuation_numbers(text)
    tokens = tokenize(text)
    tokens = remove_stopwords(tokens)
    return tokens

# Preprocess all sentences
processed_sentences = [preprocess_text(s) for s in sentences]

# Compute and print Jaccard similarity scores
print("Jaccard Similarity Scores:\n")
for i in range(len(processed_sentences)):
    for j in range(len(processed_sentences)):
        score = jaccard_similarity(
            processed_sentences[i], processed_sentences[j]
        )
        print(f"Jaccard Similarity between Sentence {i+1} and Sentence {j+1}: "
              f"{score:.3f}")
    print()
```

Jaccard Similarity between Sentence 20 and Sentence 18: 0.000

```
Jaccard Similarity between Sentence 20 and Sentence 18: 0.000
Jaccard Similarity between Sentence 20 and Sentence 19: 0.000
Jaccard Similarity between Sentence 20 and Sentence 20: 1.000
```

Close

WordNet-based Semantic Similarity

Students must:
• use WordNet synsets

• compute similarity (path / Wu-Palmer etc.)

• apply to at least 10 sentence pairs

They must discuss:
words that are different but meaningfully related.

Example:
“doctor” and “physician” become similar.

WordNet-based Semantic Similarity

Students must: • use WordNet synsets

• compute similarity (path / Wu-Palmer etc.)

• apply to at least 10 sentence pairs

They must discuss: words that are different but meaningfully related.

Example: “doctor” and “physician” become similar.

```
from nltk.corpus import wordnet as wn

def get_synsets(tokens):
    """
    Converts tokens into WordNet synsets.
    Uses the first synset as a simple heuristic.
    """
    synsets = []
    for word in tokens:
        syn = wn.synsets(word)
        if syn:
            synsets.append(syn[0])
    return synsets


def wordnet_similarity(tokens1, tokens2):
    """
    Computes semantic similarity using Wu-Palmer similarity.
    """
    synsets1 = get_synsets(tokens1)
    synsets2 = get_synsets(tokens2)

    scores = []
    for s1 in synsets1:
```

```
        for s2 in synsets2:
            sim = s1.wup_similarity(s2)
            if sim is not None:
                scores.append(sim)

    return sum(scores) / len(scores) if scores else 0
```

```
sentence_pairs_wn = [
    ("The doctor treated the patient", "The physician cured the sick person"),
    ("A car is parked outside", "An automobile is standing outdoors"),
    ("The child is eating food", "The kid is having a meal"),
    ("He bought a new house", "He purchased a new home"),
    ("The teacher explained the lesson", "The instructor taught the class"),
    ("The cat is sleeping", "The feline is resting"),
    ("She is very angry", "She feels extremely mad"),
    ("The man is running fast", "The person is sprinting quickly"),
    ("The book is interesting", "The novel is engaging"),
    ("He repaired the vehicle", "He fixed the car")
]
```

```
print("WordNet Semantic Similarity Scores:\n")

for i, (s1, s2) in enumerate(sentence_pairs_wn, start=1):
    tokens1 = preprocess_text(s1)
    tokens2 = preprocess_text(s2)

    score = wordnet_similarity(tokens1, tokens2)

    print(f"Pair {i}:")
    print("Sentence 1:", s1)
    print("Sentence 2:", s2)
    print(f"Semantic Similarity Score: {score:.3f}\n")
```

```
WordNet Semantic Similarity Scores:

Pair 1:
Sentence 1: The doctor treated the patient
Sentence 2: The physician cured the sick person
Semantic Similarity Score: 0.359

Pair 2:
Sentence 1: A car is parked outside
Sentence 2: An automobile is standing outdoors
Semantic Similarity Score: 0.374
```

```
Pair 3:
Sentence 1: The child is eating food
Sentence 2: The kid is having a meal
Semantic Similarity Score: 0.516

Pair 4:
Sentence 1: He bought a new house
Sentence 2: He purchased a new home
Semantic Similarity Score: 0.425

Pair 5:
Sentence 1: The teacher explained the lesson
Sentence 2: The instructor taught the class
Semantic Similarity Score: 0.315

Pair 6:
Sentence 1: The cat is sleeping
Sentence 2: The feline is resting
Semantic Similarity Score: 0.334

Pair 7:
Sentence 1: She is very angry
Sentence 2: She feels extremely mad
Semantic Similarity Score: 0.389

Pair 8:
Sentence 1: The man is running fast
Sentence 2: The person is sprinting quickly
Semantic Similarity Score: 0.213

Pair 9:
Sentence 1: The book is interesting
Sentence 2: The novel is engaging
Semantic Similarity Score: 0.173

Pair 10:
Sentence 1: He repaired the vehicle
Sentence 2: He fixed the car
Semantic Similarity Score: 0.520
```